

Manual de Manteniment

PETFINDER

AITOR BAUZÁ GÓMEZ - 25/26

ÍNDEX

1. Introducció.....	3
2. Estructura de l'aplicació.....	3
3. Manteniment de la base de dades.....	3
4. Manteniment de les imatges.....	4
5. Gestió de logs.....	5
6. Monitoratge de l'aplicació.....	6
7. Actualitzacions i noves versions.....	7
8. Còpies de seguretat.....	7
9. Resolució d'incidències habituals.....	8

1. Introducció

Aquest manual descriu les tasques de manteniment necessàries per garantir el bon funcionament de l'aplicació PetFinder. Està adreçat a administradors del sistema i desenvolupadors que hagin de mantenir l'aplicació en funcionament.

2. Estructura de l'aplicació

L'aplicació es compon de tres elements principals:

Component	Ubicació	Descripció
Base de dades	PostgreSQL	Emmagatzema totes les dades d'usuaris, anuncis, missatges, etc.
Backend	Spring Boot	Servidor REST que gestiona la lògica de negoci.
Frontend	React	Interfície d'usuari que es consumeix des del navegador.
<i>Arxius pujats</i>	<i>Carpeta uploads/</i>	<i>Imatges de mascotes i perfils d'usuari.</i>

3. Manteniment de la base de dades

3.1 Neteja de registres antics

Amb el temps, la base de dades pot acumular registres obsolets. Es recomana executar periòdicament:

```
DELETE FROM ubicacio_temps_real  
WHERE timestamp < NOW() - INTERVAL '30 days';
```

```
DELETE FROM conversa  
WHERE conversa_id NOT IN (SELECT DISTINCT id_conversa FROM  
missatge);
```

3.2 Optimització de la base de dades

És recomanable executar periòdicament VACUUM i ANALYZE per optimitzar el rendiment.

3.3 Verificació d'integritat referencial

Per comprovar que no hi ha dades òrfenes:

```
SELECT * FROM anunci WHERE id_mascota IS NOT NULL  
AND id_mascota NOT IN (SELECT mascota_id FROM mascota);
```

```
SELECT * FROM mascota WHERE id_usuari IS NOT NULL  
AND id_usuari NOT IN (SELECT usuari_id FROM usuari);
```

4. Manteniment de les imatges

4.1 Estructura de carpetes

Les imatges pujades pels usuaris s'emmagatzemen a:

```
uploads/  
├─ mascotes/      # Imatges de mascotes  
└─ perfils/       # Imatges de perfil d'usuari
```

4.2 Neteja d'imatges òrfenes

Amb el temps, poden quedar imatges a la carpeta uploads/ que no estiguin associades a cap registre a la base de dades. Per netejar-les:

```
SELECT url FROM imatge;
```

(manualment, cal revisar les carpetes i eliminar els fitxers que no apareguin a la consulta anterior)

4.3 Reducció de la mida de les imatges

Per optimitzar l'espai i la càrrega, es recomana comprimir les imatges abans de pujar-les. Es pot implementar una compressió automàtica al backend.

4.4 Còpia de seguretat de les imatges

Les imatges són part important de l'aplicació. Es recomana incloure la carpeta uploads/ en les còpies de seguretat periòdiques.

5. Gestió de logs

5.1 Configuració dels logs

Per activar l'emmagatzematge de logs en un fitxer, afegeix al application.properties:

```
logging.file.name=logs/petfinder.log
logging.pattern.file=%d{yyyy-MM-dd HH:mm:ss} - %msg%n
```

5.2 Rotació de logs

Per evitar que els logs ocupin massa espai, configura la rotació:

```
logging.logback.rollingpolicy.max-history=7
logging.logback.rollingpolicy.max-file-size=10MB
```

5.3 Consulta de logs

Els logs es troben a la carpeta logs/ del backend. Per consultar els errors recents llança la següent comanda:

```
tail -100 logs/petfinder.log | grep ERROR
```

5.4 Nivells de log recomanats

Entorn	Nivell
Desenvolupament	DEBUG
Producció	INFO o WARN

6. Monitoratge de l'aplicació

6.1 Endpoint de salut (Healthcheck)

Es pot afegir un endpoint bàsic per comprovar que l'aplicació funciona:

```
@RestController
public class HealthController {
    @GetMapping("/health")
    public ResponseEntity<String> health() {
        return ResponseEntity.ok("OK");
    }
}
```

6.2 Monitoratge extern

Es pot utilitzar eines com UptimeRobot o Cronjobs per verificar periòdicament que l'endpoint /health respon correctament.

6.3 Comprovar l'estat del backend (terminal)

```
curl http://localhost:9090/health
(ha de retornar OK)
```

6.4 Comprovar l'estat de la base de dades (terminal)

```
docker exec -it petfinder-db pg_isready -U postgres
```

7. Actualitzacions i noves versions

7.1 Actualització del backend

```
cd petfinder-api
git pull
./mvnw clean package
./mvnw spring-boot:run
```

7.2 Actualització del frontend

```
cd petfinder-front
git pull
npm install
npm run build
```

7.3 Migracions de base de dades

Quan s'afegeixin noves entitats o camps, Spring Boot actualitzarà automàticament l'esquema si **ddl-auto és update**.

En producció, es recomana utilitzar validate i aplicar els canvis manualment.

8. Còpies de seguretat

8.1 Còpia de seguretat de la base de dades

```
pg_dump -U postgres -d petfinder > backup_petfinder_$(date +%Y%m%d).sql
```

8.2 Còpia de seguretat de les imatges

```
tar -czf uploads_backup_$(date +%Y%m%d).tar.gz uploads/
```

8.3 Programar còpies automàtiques (cron)

Exemple per executar còpies cada dia a les 2 de la matinada:

```
0 2 * * * pg_dump -U postgres -d petfinder >
/backups/petfinder_$(date +%Y%m%d).sql
```

8.4 Restauració d'una còpia

```
psql -U postgres -d petfinder < backup_petfinder_20260101.sql
```

9. Resolució d'incidències habituals

9.1 L'aplicació no arrenca

Causa possible:

- Port ocupat
- Base de dades no accessible.

Solució:

- Comprovar si el port 9090 està ocupat:

```
lsof -i :9090
```

- Comprovar que PostgreSQL està en execució:

```
docker ps | grep petfinder-db
```

9.2 Les imatges no es carreguen

Causa possible: Permisos incorrectes a la carpeta uploads/.

Solució:

```
chmod -R 755 uploads/
```


9.3 L'usuari no pot iniciar sessió

Causa possible:

- Contrasenya incorrecta.
- Usuari no registrat.

Solució - Comprovar a la base de dades:

```
SELECT usuari_id, email, rol FROM usuari WHERE email =  
'usuari@example.com';
```

9.4 El mapa no carrega

Causa possible: Problema de connexió a OpenStreetMap o clau d'API incorrecta.

Solució - Comprovar que el frontend té connexió a Internet i que la URL de Leaflet és correcta.

9.5 L'aplicació va lenta

Causa possible: Manca d'índexs a la base de dades o moltes dades.

Solució:

```
CREATE INDEX idx_anunci_data ON anunci(data);  
CREATE INDEX idx_missatge_conversa ON missatge(id_conversa);  
CREATE INDEX idx_ubicacio_mascota ON ubicacio_temps_real(mascota_id);
```

9.6 Error de memòria (OutOfMemory)

Causa possible: Manca de memòria assignada al backend.

Solució:

- Augmentar la memòria disponible per a Spring Boot:

```
./mvnw spring-boot:run  
-Dspring-boot.run.jvmArguments="-Xmx512m"
```